Módulo 6 — Eventos Académicos

Vistas del módulo y arquitectura elegida, con sus integraciones a otros módulos y servicios externos.

Desarrollo de Aplicaciones II · UADE · 2.º cuatrimestre 2026
Repositorio `uadenet-eventos` · Documento generado el 11 de noviembre de 2026

Contenido

1. Vistas del módulo

#	Vista	Ruta	Perfil
1	Ingreso	`/`	Todos
2	Cartelera de eventos	`/cartelera`	Todos
3	Detalle del evento	`/eventos/[id]`	Todos
4	Mis inscripciones	`/mis-inscripciones`	Todos
5	Cuenta institucional	`/cuenta`	Todos
6	Avisos	`/avisos`	Todos
7	Gestión de eventos	`/gestion`	Administrativo
8	Nuevo evento	`/eventos/nuevo`	Administrativo
9	Detalle con panel de gestión	`/eventos/[id]`	Creador del evento
10	Editar evento	`/eventos/[id]/editar`	Creador del evento
11	Asistencia en vivo	`/asistencia`	Creador del evento
12	Estado de la API	`/health`	Técnica

2. Arquitectura

Stack y estructura · contexto dentro de UADEnet · capas de la API · despliegue.

3. Integraciones con terceros

CORE · Backoffice Administrativo · Analítica Institucional · Portales · Supabase · Vercel · librerías de QR · almacenamiento de imágenes.

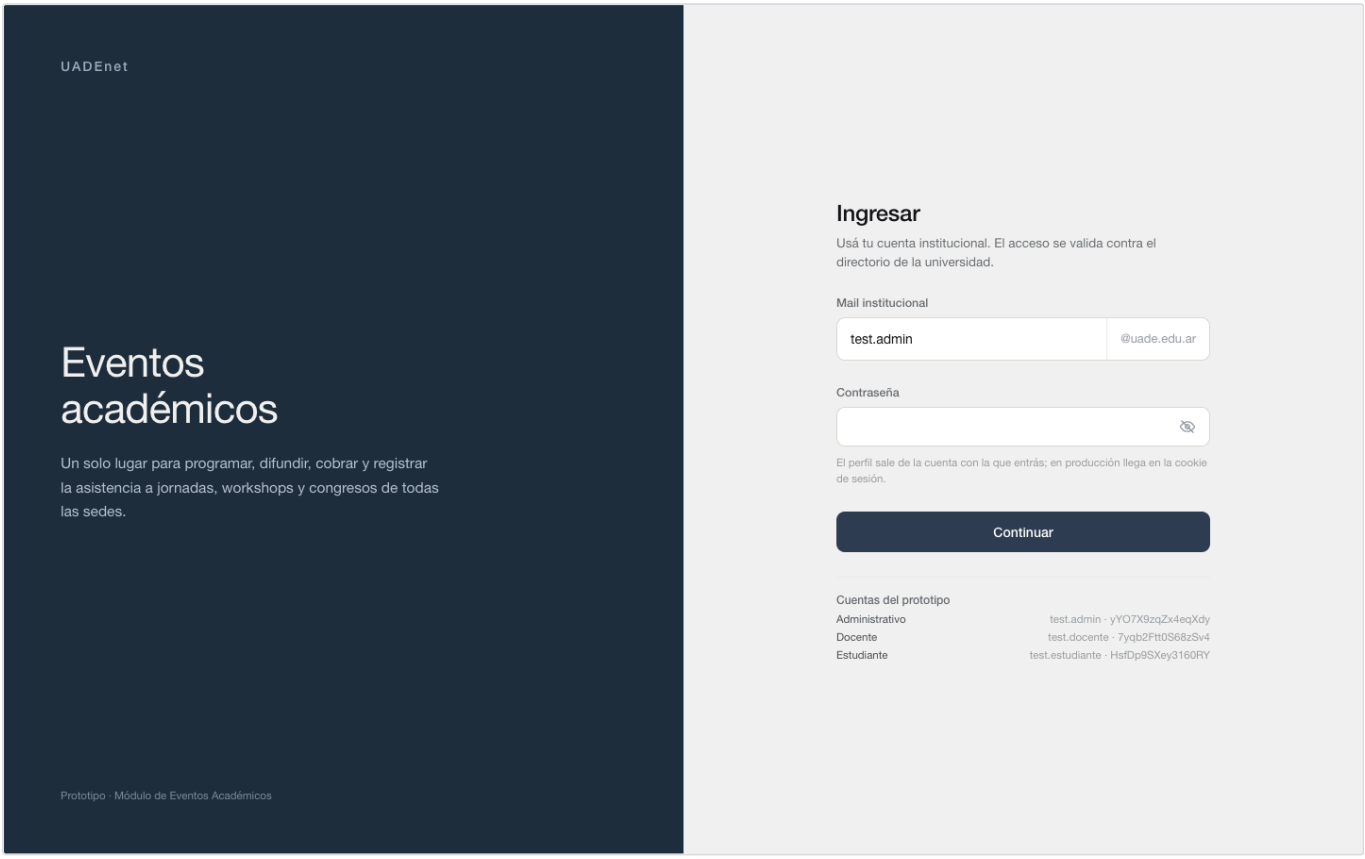
4. Decisiones de arquitectura

Los 17 ADRs del repositorio agrupados por tema (stack, base de datos, ramas, Backoffice y comportamiento del sistema): qué se eligió, qué se descartó y por qué. Cierra con los pendientes de integración.

> **Cómo se tomaron las capturas.** La aplicación corre en local (`pnpm dev`) contra una base Postgres propia, sin `CORE_BASE_URL` ni `BACKOFFICE_BASE_URL` : en ese modo la API monta el CORE y el Backoffice simulados, que responden con los mismos contratos que los reales. Los eventos, inscripciones, la asistencia y la reprogramación se cargaron a través de la propia API. Se entró con las cuentas de demo del administrativo (Mariana Ibarra) y del estudiante (Tomás Vidal). Resolución 1440 × 900.

1. Ingreso

/ Sin sesión



QUÉ MUESTRA

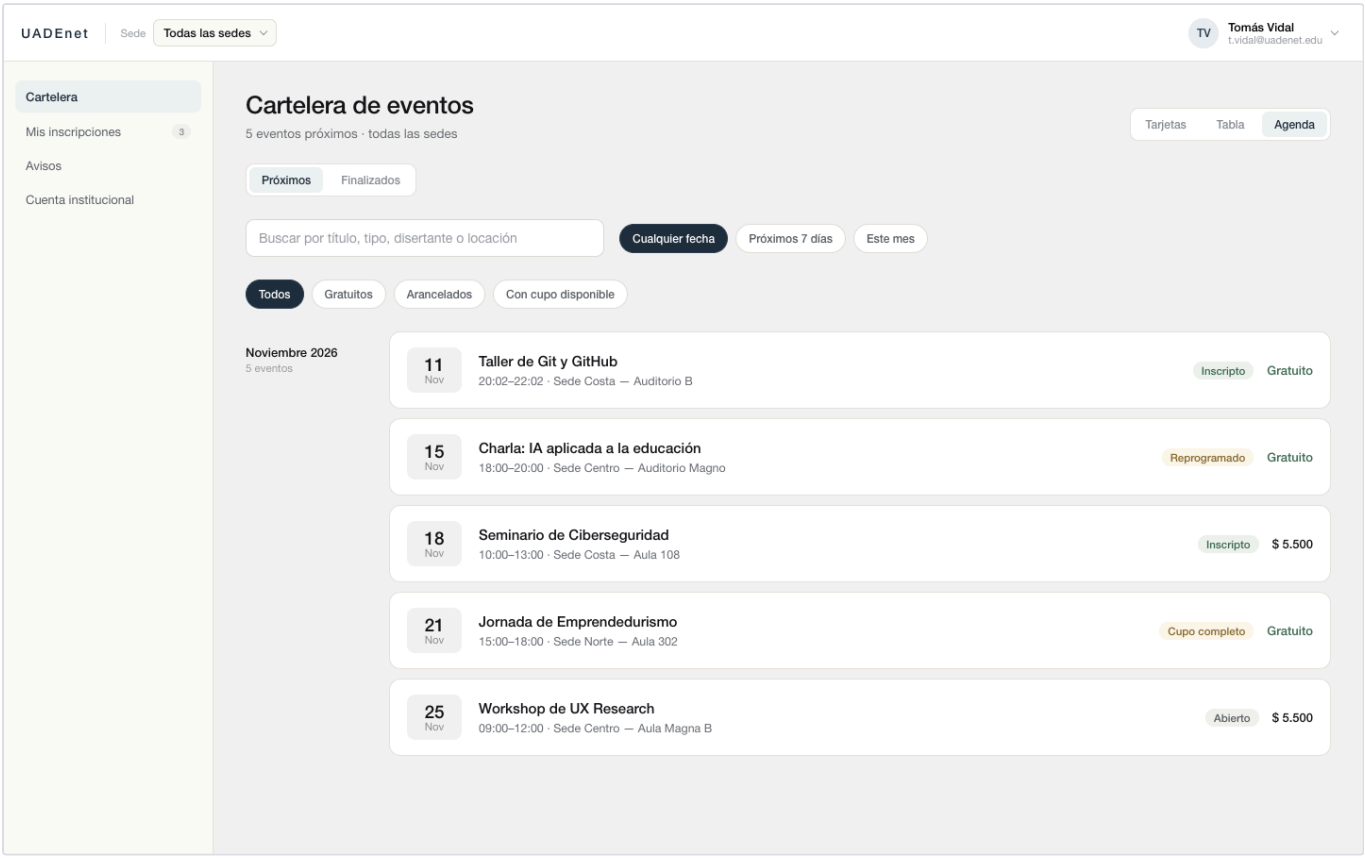
Login con mail institucional y contraseña. El perfil (administrativo, docente o estudiante) no se elige: sale de los roles que trae la cuenta. Al pie se listan las cuentas de prueba del prototipo.

INTEGRACIÓN

`POST /api/v1/auth/login` en la API, que lo reenvía a CORE (IAM). CORE devuelve un JWT RS256; el portal lo guarda en la sesión del navegador y lo manda como `Bearer` en cada request.

2. Cartelera de eventos

`/cartelera` Todos los perfiles



QUÉ MUESTRA

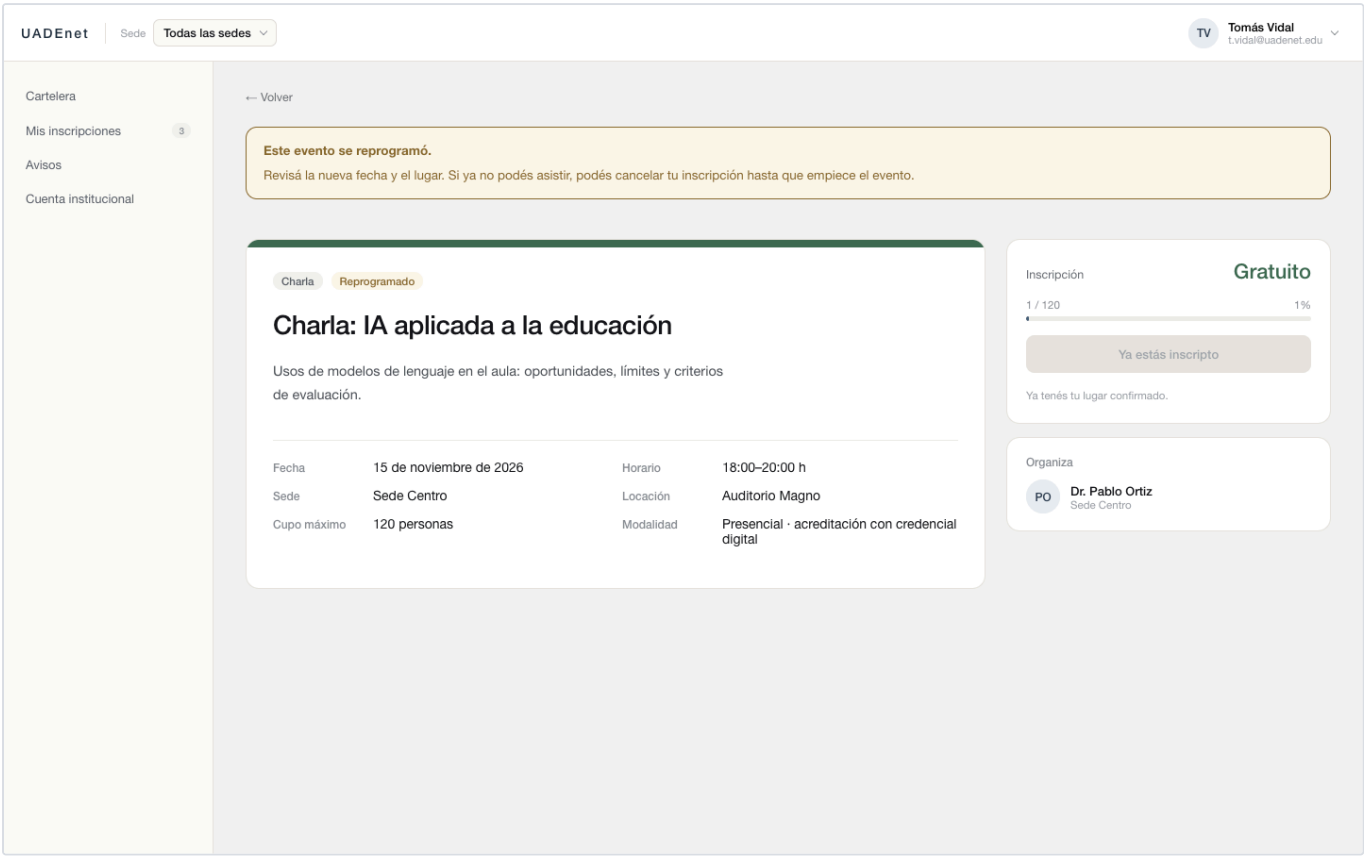
Eventos próximos y finalizados de todas las sedes o de una. Búsqueda por título, tipo, disertante o locación; filtros por período, gratuitos/arancelados y cupo disponible. Tres vistas: tarjetas, tabla y agenda (en la captura). Cada evento muestra su estado: inscripto, reprogramado, cupo completo o abierto.

INTEGRACIÓN

`GET /api/v1/eventos`, `/locaciones`, `/sedes` y `/tarifas`. Las sedes y locaciones vienen del catálogo de Backoffice sincronizado en la base; el precio de los arancelados, de la tarifa vigente de Backoffice en caché.

3. Detalle del evento

`/eventos/[id]` Todos los perfiles (vista del estudiante)



QUÉ MUESTRA

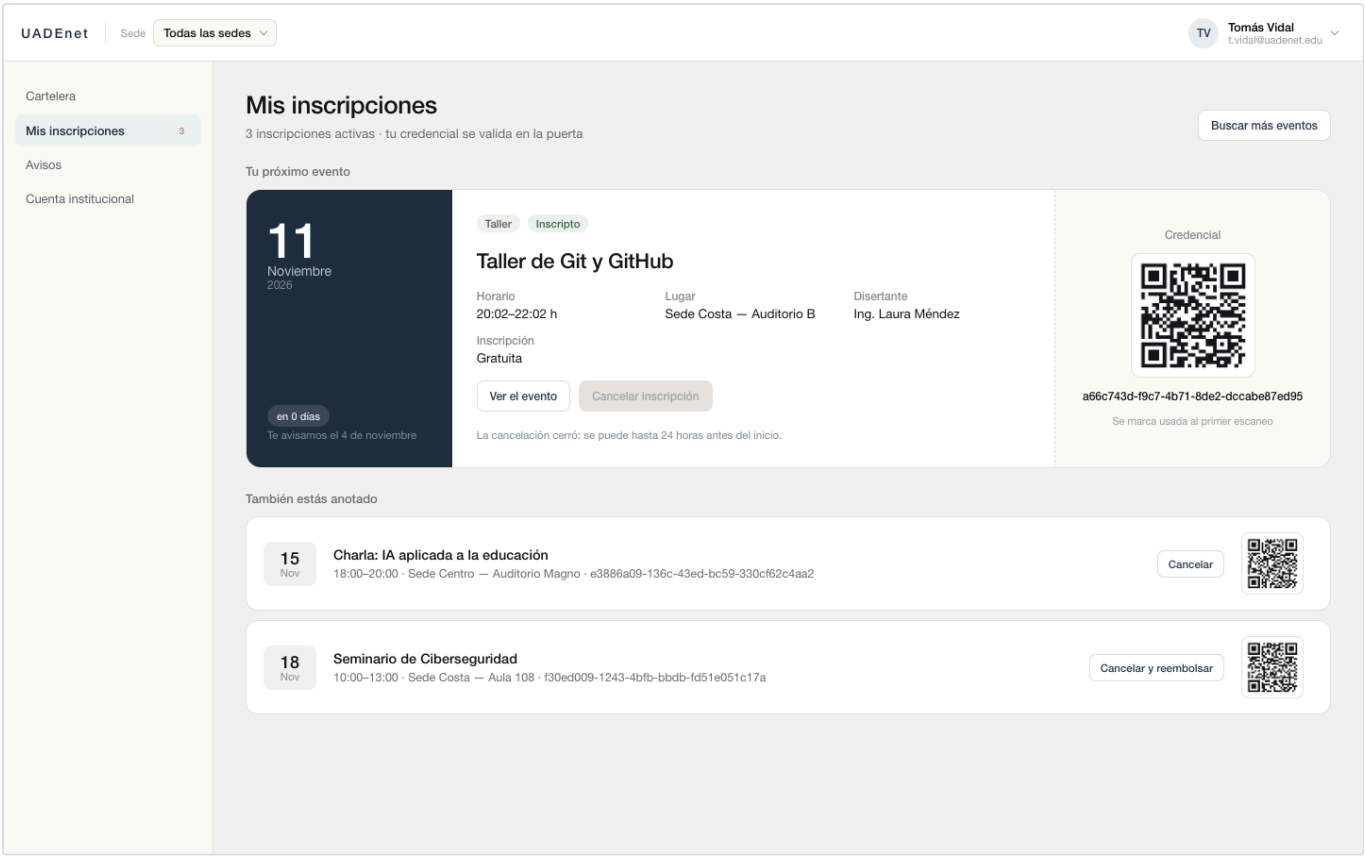
Datos completos del evento, ocupación del cupo y acción de inscripción. Si es arancelado, el checkout muestra el saldo antes y después del cobro. Si el evento se reprogramó aparece el banner, y la persona puede cancelar su inscripción sin el límite de 24 horas.

INTEGRACIÓN

`GET /api/v1/eventos/:id` y `/cupo`. Inscribirse es `POST /api/v1/inscripciones`: valida cupo y choque de horario, y en los pagos debita la billetera de CORE con clave de idempotencia. Emite `inscripcion.registrada` a Analítica.

4. Mis inscripciones

`/mis-inscripciones` Todos los perfiles



QUÉ MUESTRA

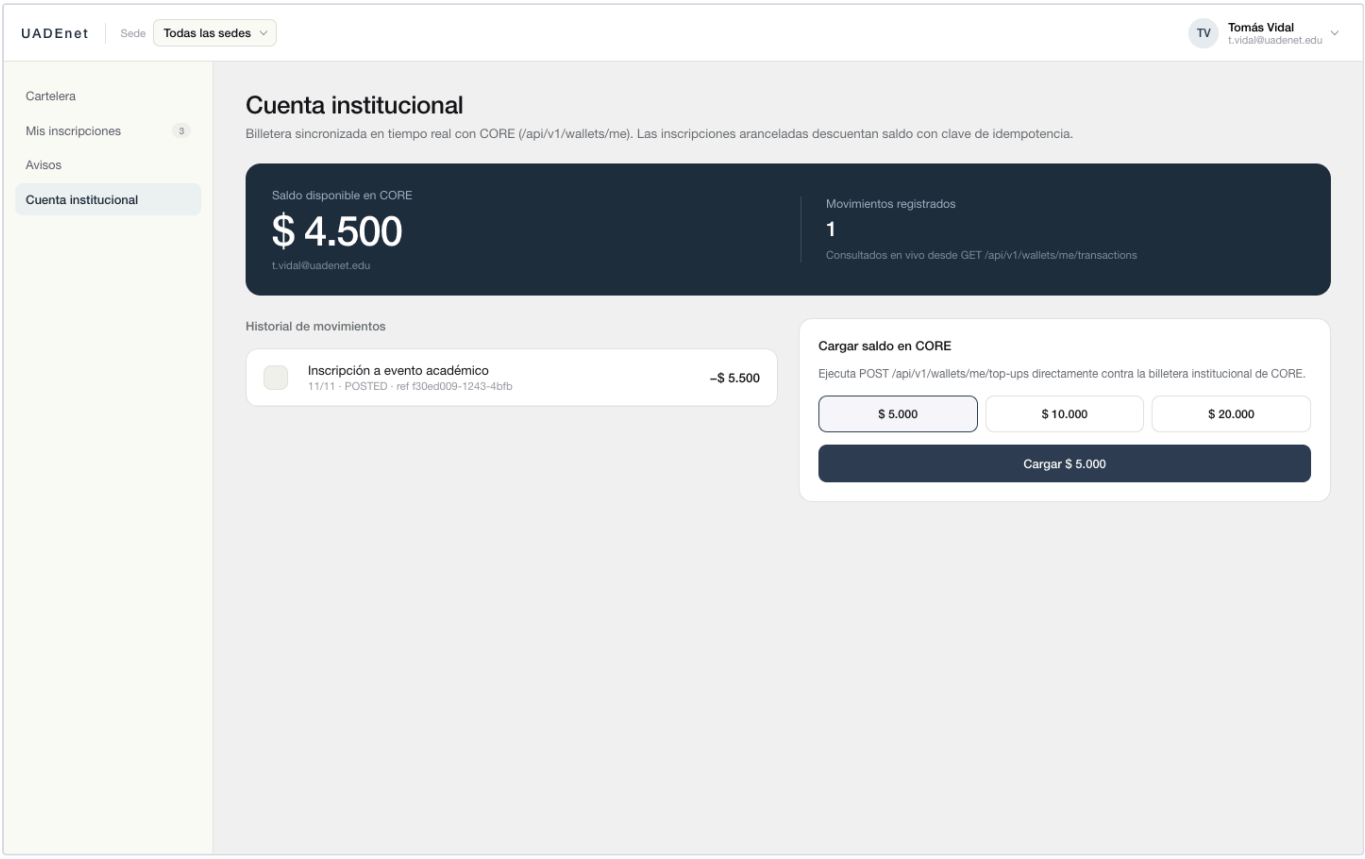
El próximo evento destacado, con la credencial QR que se presenta en la puerta y la fecha del recordatorio automático (7 días antes). Debajo, el resto de las inscripciones activas y la acción de cancelar (con reembolso si fue paga), habilitada hasta 24 horas antes del inicio.

INTEGRACIÓN

`GET /api/v1/inscripciones` y `DELETE /api/v1/inscripciones/:id`. La cancelación de una inscripción paga busca el cobro en CORE por referencia y lo revierte. El QR se genera en el navegador con la librería `qrcode` a partir del id de la inscripción.

5. Cuenta institucional

`/cuenta` Todos los perfiles



QUÉ MUESTRA

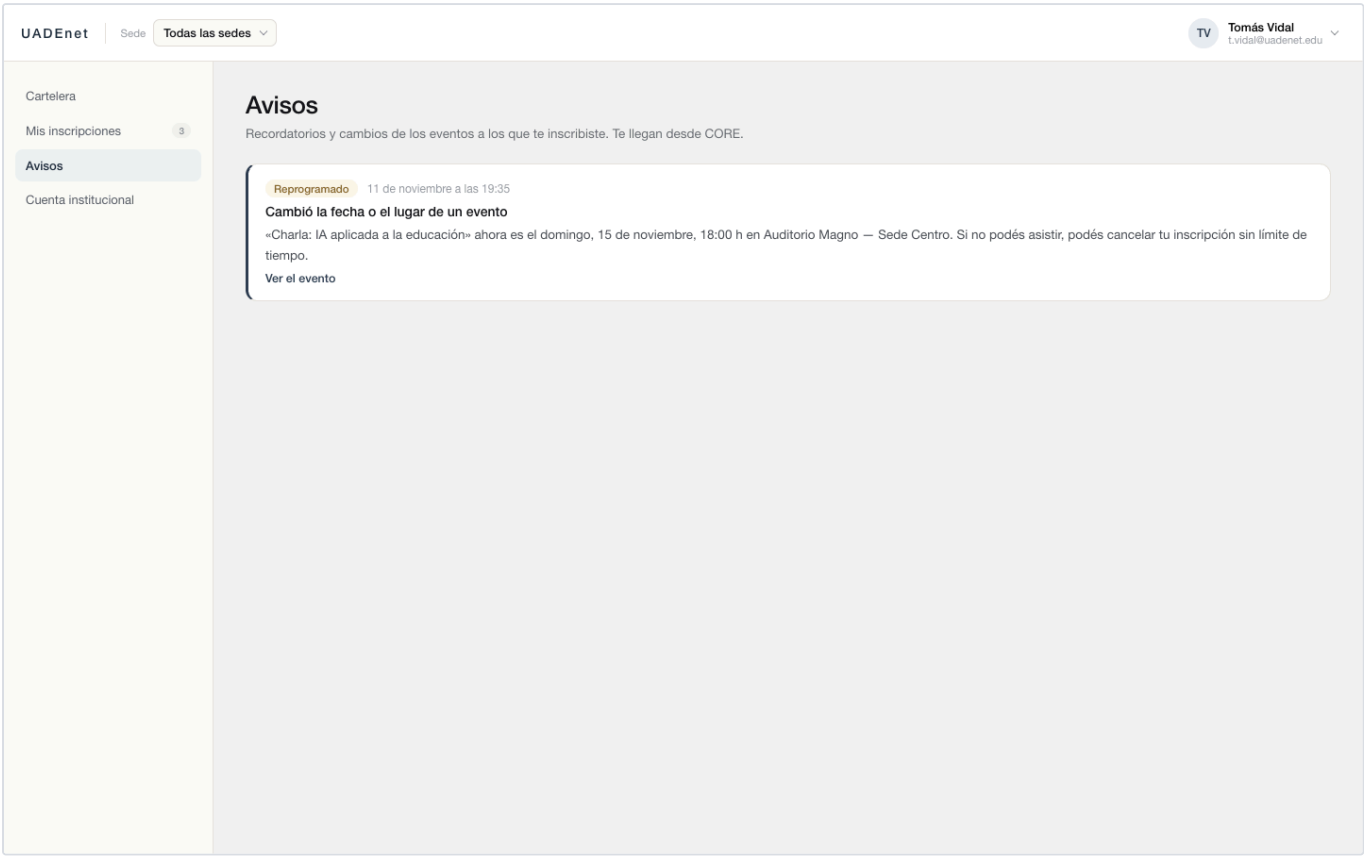
Saldo disponible en la billetera institucional, historial de movimientos (cobros de inscripciones y reembolsos) y recarga de prueba.

INTEGRACIÓN

Todo es de CORE, dueño de las billeteras: `GET /api/v1/usuarios/me/saldo`, `/me/movimientos` y `POST /me/recargas`, que la API traduce a `/api/v1/wallets/me`, `/wallets/me/transactions` y `/wallets/me/top-ups`. El módulo no guarda saldos.

6. Avisos

`/avisos` Todos los perfiles



QUÉ MUESTRA

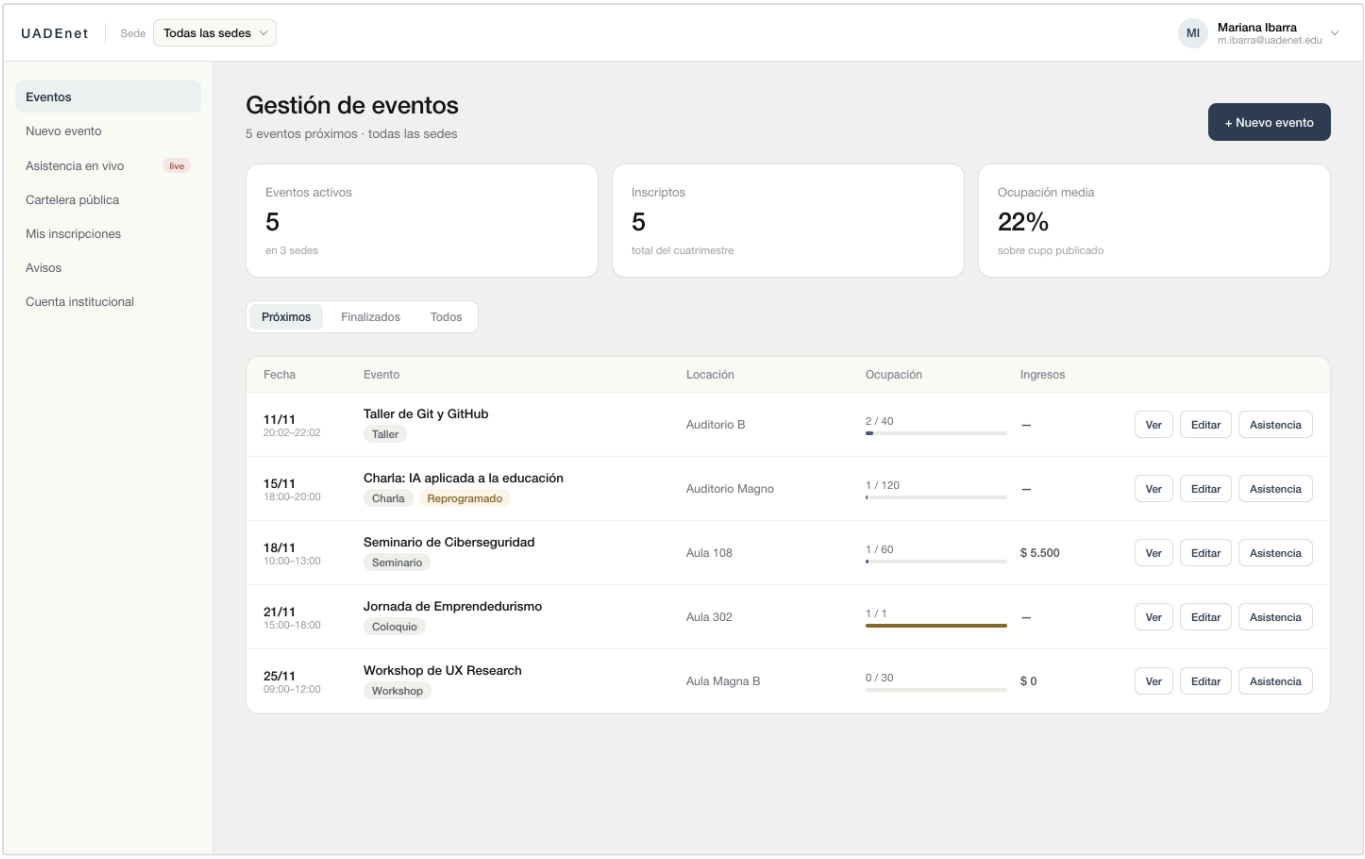
Recordatorios y cambios de los eventos a los que la persona se inscribió: recordatorio a 7 días, reprogramación (en la captura) y cancelación por el organizador, con link al evento.

INTEGRACIÓN

Las notificaciones las guarda y reparte CORE. La API las publica con `POST /internal/v1/notifications` (`EVENT_REMINDER`, `EVENT_RESCHEDULED`, `EVENT_CANCELLED`) y esta vista las lee con `GET /api/v1/usuarios/me/notificaciones`. Los portales las muestran en su campanita.

7. Gestión de eventos

`/gestion` Administrativo



QUÉ MUESTRA

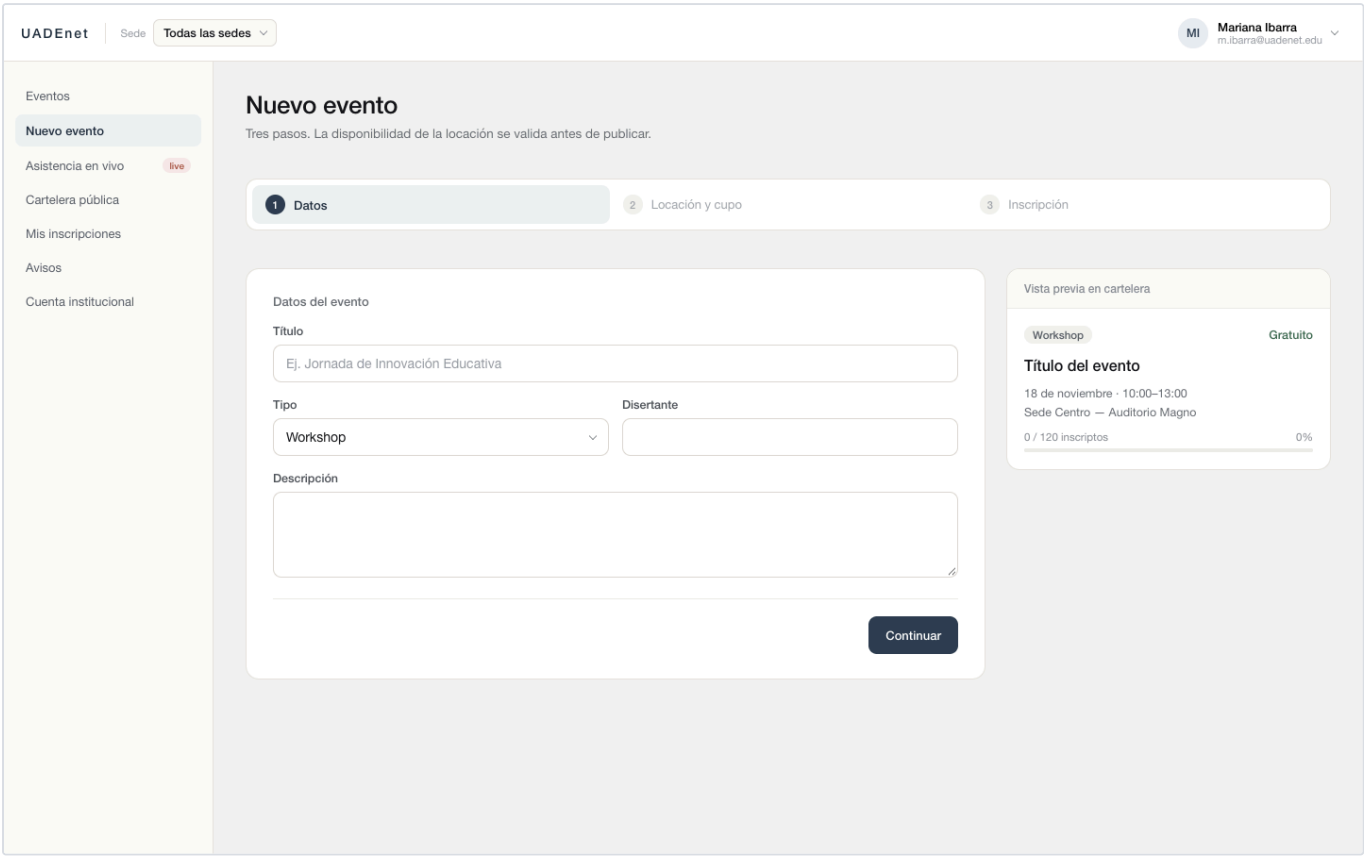
Tablero del administrativo: eventos activos, inscriptos totales y ocupación media. Tabla de eventos próximos, finalizados o todos, con ocupación, ingresos y accesos a ver, editar y tomar asistencia.

INTEGRACIÓN

Misma lectura que la cartelera (`GET /api/v1/eventos`). La pantalla solo está en el menú de quien tiene el rol administrativo o el permiso `campus-events:manage` de CORE.

8. Nuevo evento

`/eventos/nuevo` Administrativo



QUÉ MUESTRA

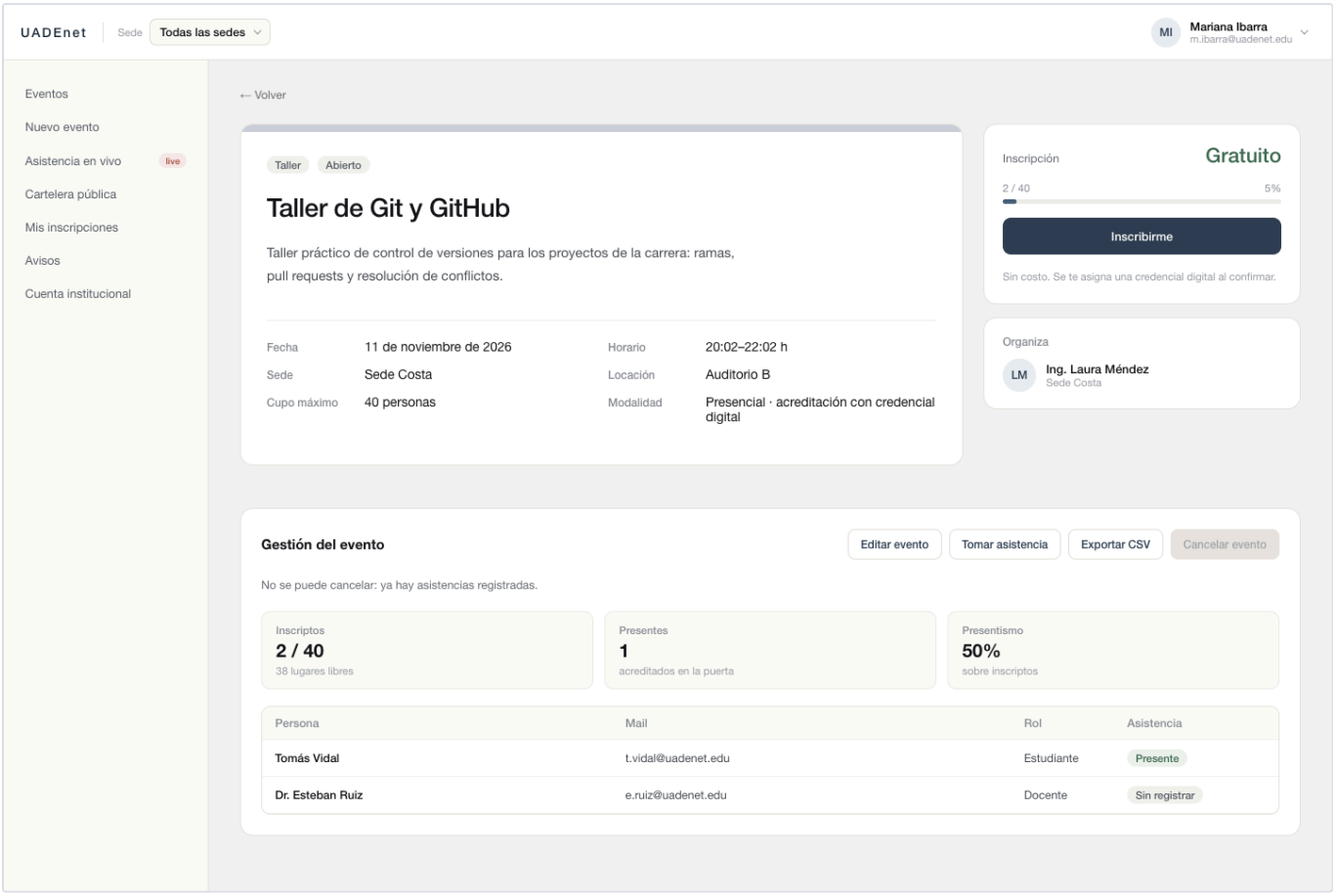
Alta en tres pasos: datos, locación y cupo, inscripción (gratuita o arancelada). Al costado, la vista previa de cómo se va a ver en la cartelera. En el paso 2, una línea de ocupación muestra si la locación ya está tomada en ese horario y ofrece alternativas libres.

INTEGRACIÓN

`POST /api/v1/eventos`, validado con el schema compartido de `@repo/contracts`. La API rechaza el solapamiento de locación y, si es arancelado, exige una tarifa vigente de Backoffice. Emite `evento.creado` a Analítica.

9. Detalle con panel de gestión

`/eventos/[id]` Creador del evento



QUÉ MUESTRA

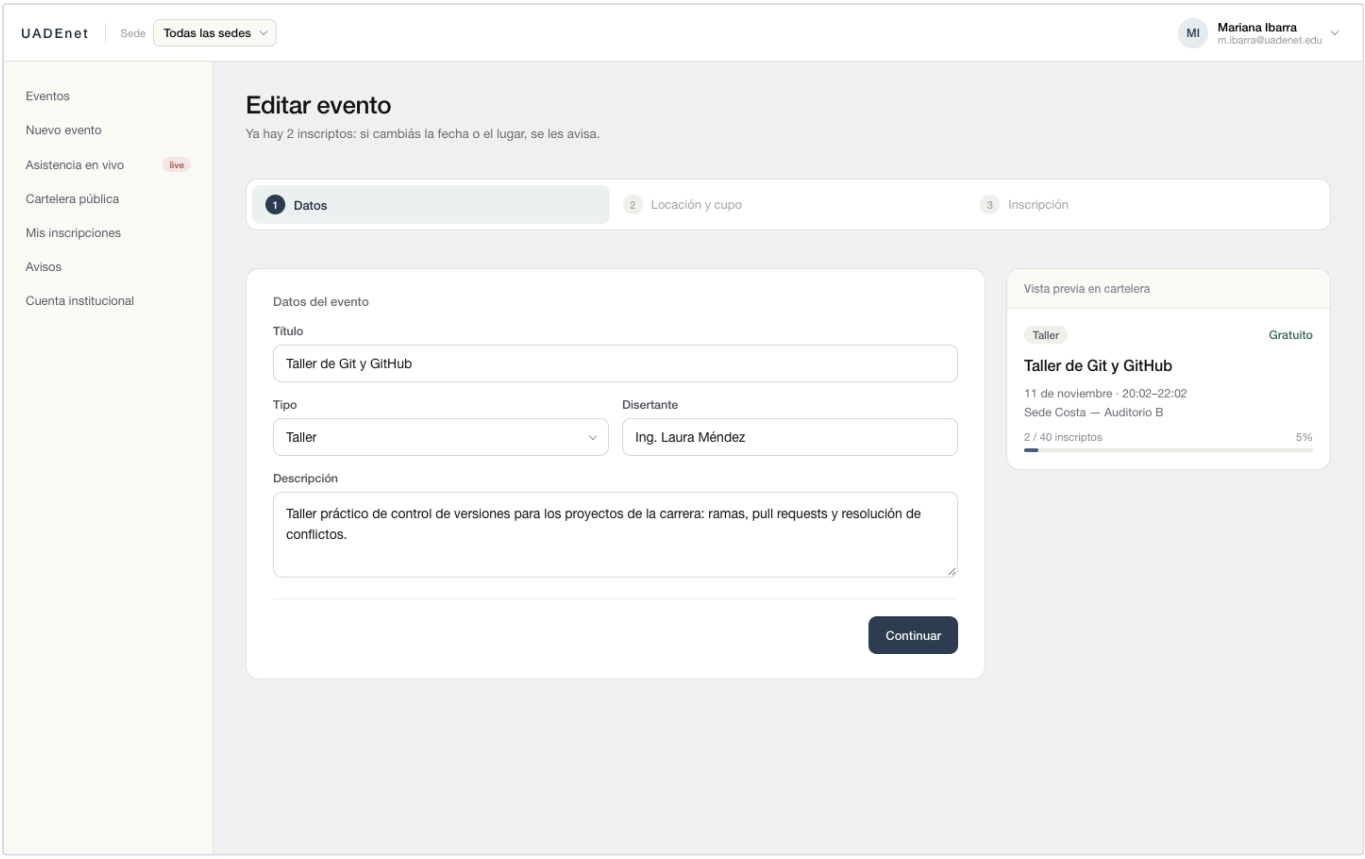
Para quien creó el evento, el detalle suma el panel de gestión: inscriptos, presentes y presentismo, la lista de inscriptos con su asistencia, exportación a CSV, y las acciones de editar, tomar asistencia y cancelar. Cancelar se bloquea con el motivo visible (en la captura: ya hay asistencias registradas).

INTEGRACIÓN

`GET /api/v1/asistencia?eventoId=` y `GET /api/v1/eventos/:id/impacto` (cuántos inscriptos y cuánto se reembolsaría). `POST /api/v1/eventos/:id/cancelacion` revierte cada cobro en CORE, avisa a los inscriptos y emite `evento.cancelado`.

10. Editar evento

`/eventos/[id]/editar` Creador del evento



QUÉ MUESTRA

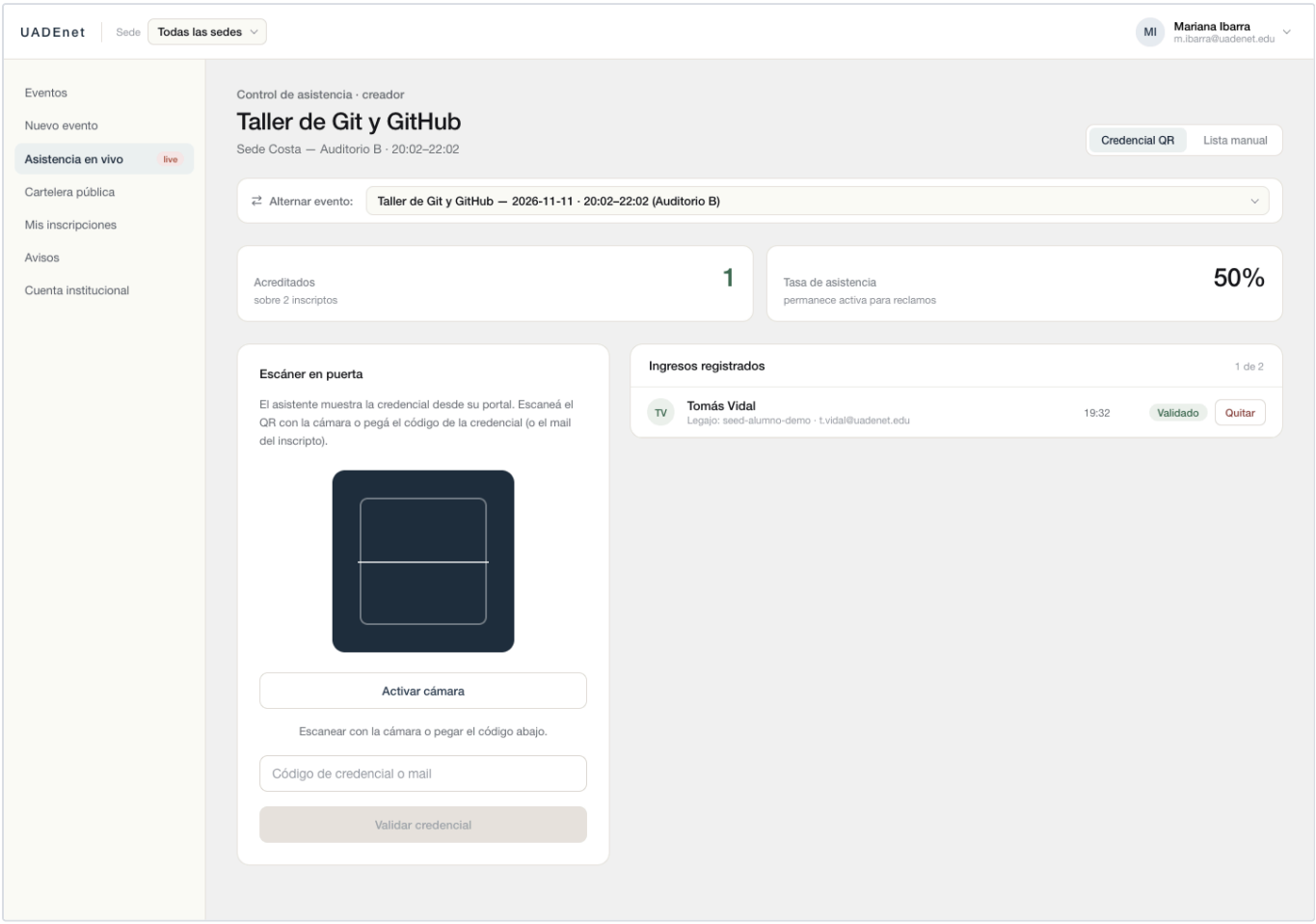
El mismo asistente del alta, precargado. Avisa cuántos inscriptos hay y bloquea con el motivo lo que no se puede cambiar: bajar el cupo por debajo de los inscriptos, o cambiar el precio con inscriptos. Cambiar fecha o locación muestra "antes → después" y "Guardar y avisar a N".

INTEGRACIÓN

`PATCH /api/v1/eventos/:id`. Una reprogramación revalida locación y choques de los inscriptos, notifica por CORE (`EVENT_RESCHEDULED`) y emite `evento.modificado` a Analítica.

11. Asistencia en vivo

`/asistencia` Creador del evento



QUÉ MUESTRA

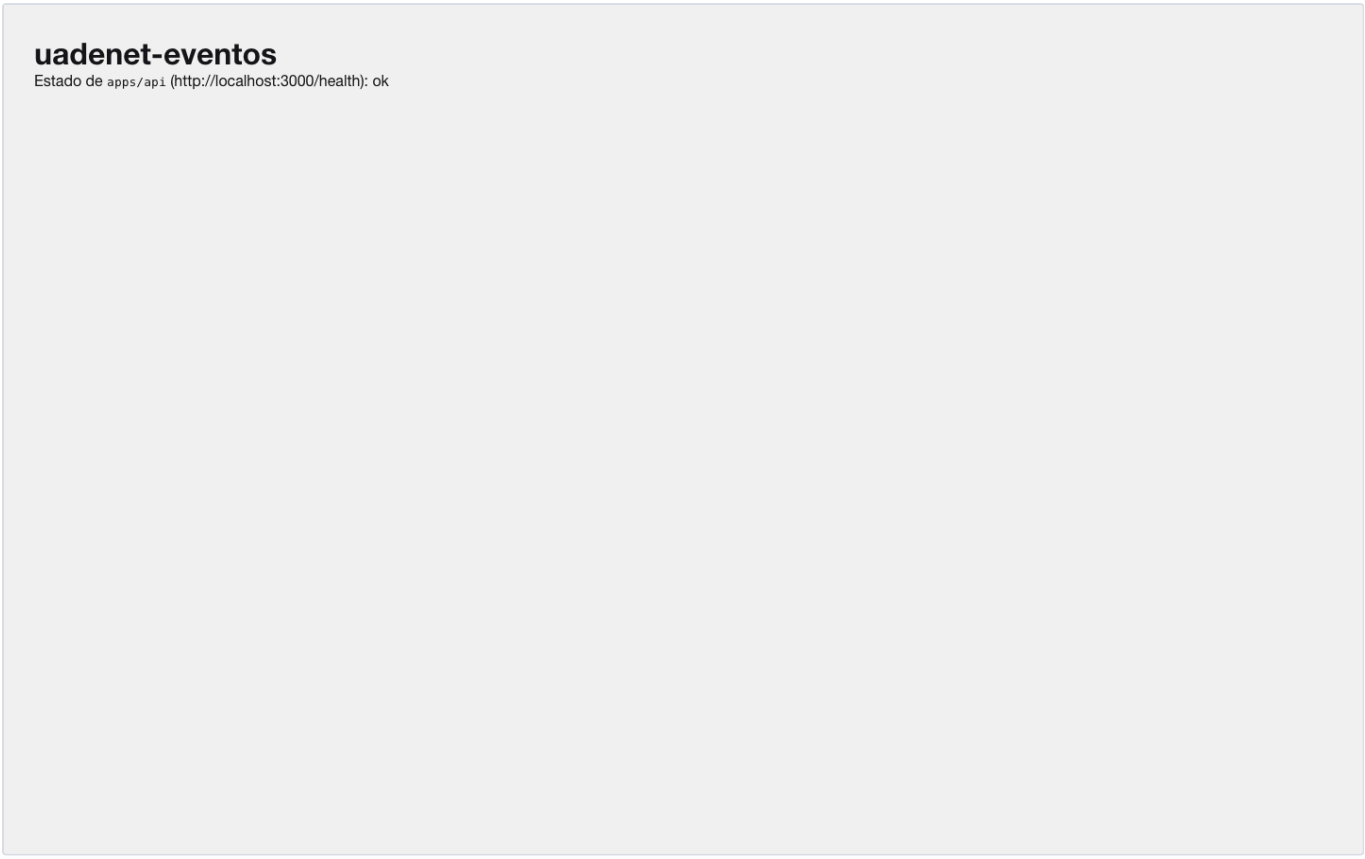
Control de asistencia en la puerta, habilitado desde una hora antes del inicio. Dos modos: escaneo del QR de la credencial con la cámara del dispositivo (o pegando el código o el mail) y lista manual con búsqueda. Contadores de acreditados y tasa de asistencia; un ingreso se puede revertir.

INTEGRACIÓN

`POST /api/v1/asistencia` y `DELETE /api/v1/asistencia/:inscripcionId`; emite `asistencia.confirmada` a Analítica. El QR se decodifica en el navegador con `jsQR` sobre la cámara (`getUserMedia`): ninguna imagen sale del dispositivo.

12. Estado de la API

`/health` Vista técnica, sin sesión



QUÉ MUESTRA

Página de diagnóstico: confirma desde el portal que la API responde y a qué URL apunta (`NEXT_PUBLIC_API_URL`). Se usa al desplegar y antes de una demo.

INTEGRACIÓN

`GET /health` de `apps/api` . El contrato completo de la API está documentado en Swagger en `/api/docs` .

Arquitectura elegida

Stack

Capa	Tecnología	Por qué
Frontend	Next.js 16 (App Router), React 19, CSS Modules	Portal del módulo; los estilos usan los tokens del sistema de diseño (ADR 0010).
Backend	NestJS 11 + Swagger	App separada del front, con módulos por feature, guards, pipes y OpenAPI generado (ADR 0003). El Swagger es el contrato con los otros grupos.
ORM	Drizzle	Sin paso de generación, liviano en serverless y agnóstico del proveedor de Postgres (ADR 0004).
Base de datos	Supabase (PostgreSQL)	Postgres administrado con free tier; un proyecto compartido por el equipo (ADR 0011).
Contratos	Zod en `packages/contracts`	Un solo schema valida requests en la API y formularios en el portal, sin duplicar DTOs (ADR 0005).
Monorepo	Turborepo + pnpm workspaces	Web, API y paquetes compartidos en un repo, con build y caché por paquete (ADR 0002).
Deploy	Vercel (dos proyectos) + Vercel Cron	Deploy por rama (`dev` y `main`) sin infraestructura propia (ADR 0009, 0016).

Estructura del monorepo

`apps/web`	Portal Next.js. Una carpeta por pantalla en `app/` ; lo compartido en `components/` y la lógica sin JSX (estado del evento, conflictos, formato) en `lib/` .
`apps/api`	NestJS por feature: `modules/` (eventos, inscripciones, asistencia, locaciones, usuarios, auth, cron), `common/` (guards, pipes e integraciones con CORE, Backoffice y Analítica) y `workers/` (recordatorios).
`packages/contracts`	Schemas Zod y tipos compartidos.
`packages/db`	Schema de Drizzle (`usuarios` , `locaciones` , `eventos` , `inscripciones` , `asistencias`), migrations y seed.
`packages/env`	Validación de variables de entorno al arrancar.

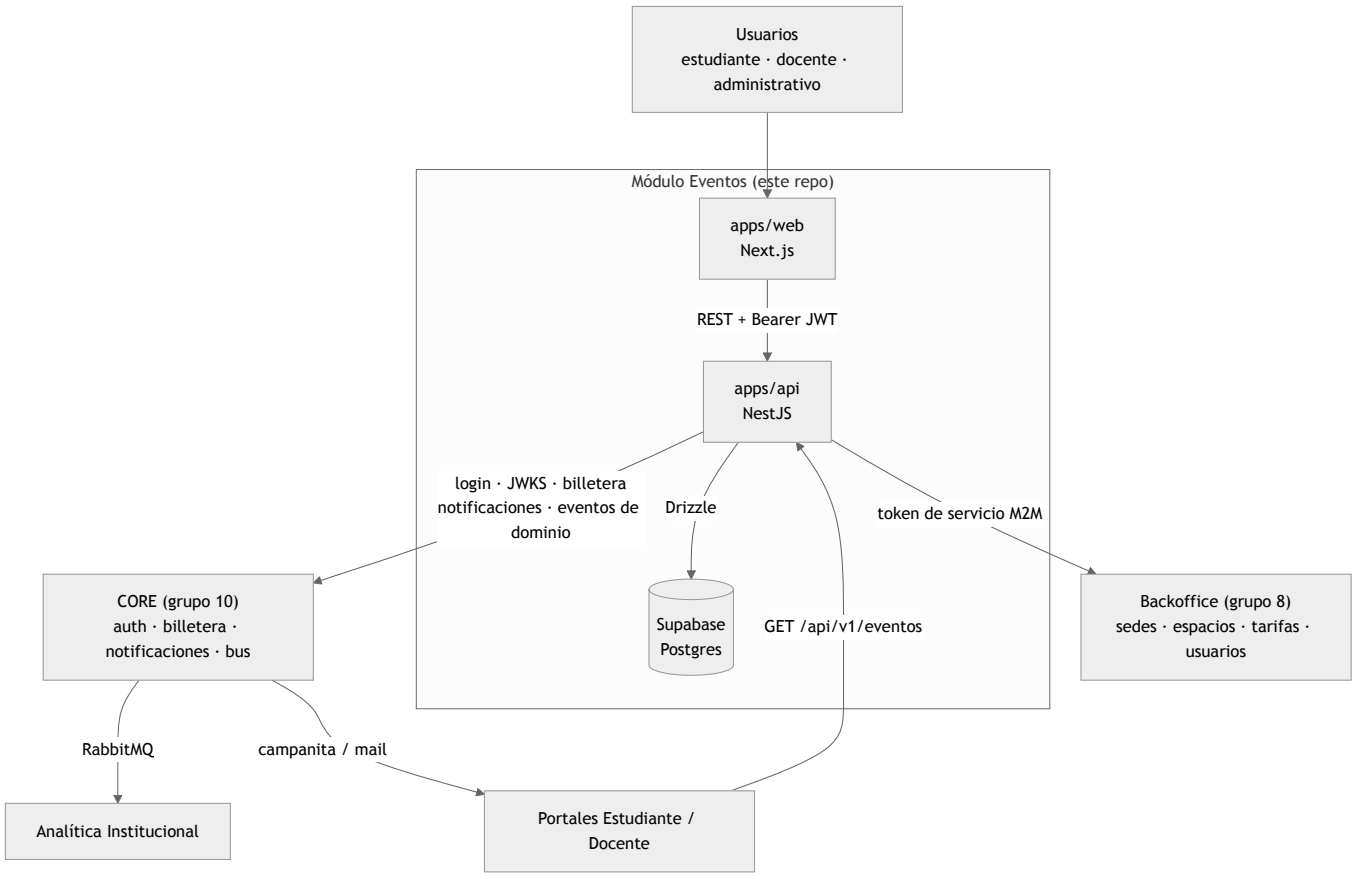
Flujo de un request

El portal nunca consulta la base directo. Todo pasa por la API, que valida el JWT de CORE y el schema del request antes de llegar a la lógica de negocio:

`Portal (web) → CORE (gateway + auth) → apps/api (NestJS) → packages/db (Drizzle) → Supabase (Postgres)`

Dentro de la API, cada módulo tiene tres capas: **controller** (recibe y valida, sin lógica), **service** (reglas: locación libre, sin choque de horario por persona, cupo, quién edita o cancela) y **Drizzle** (acceso a datos).

Contexto: el módulo dentro de UADEnet

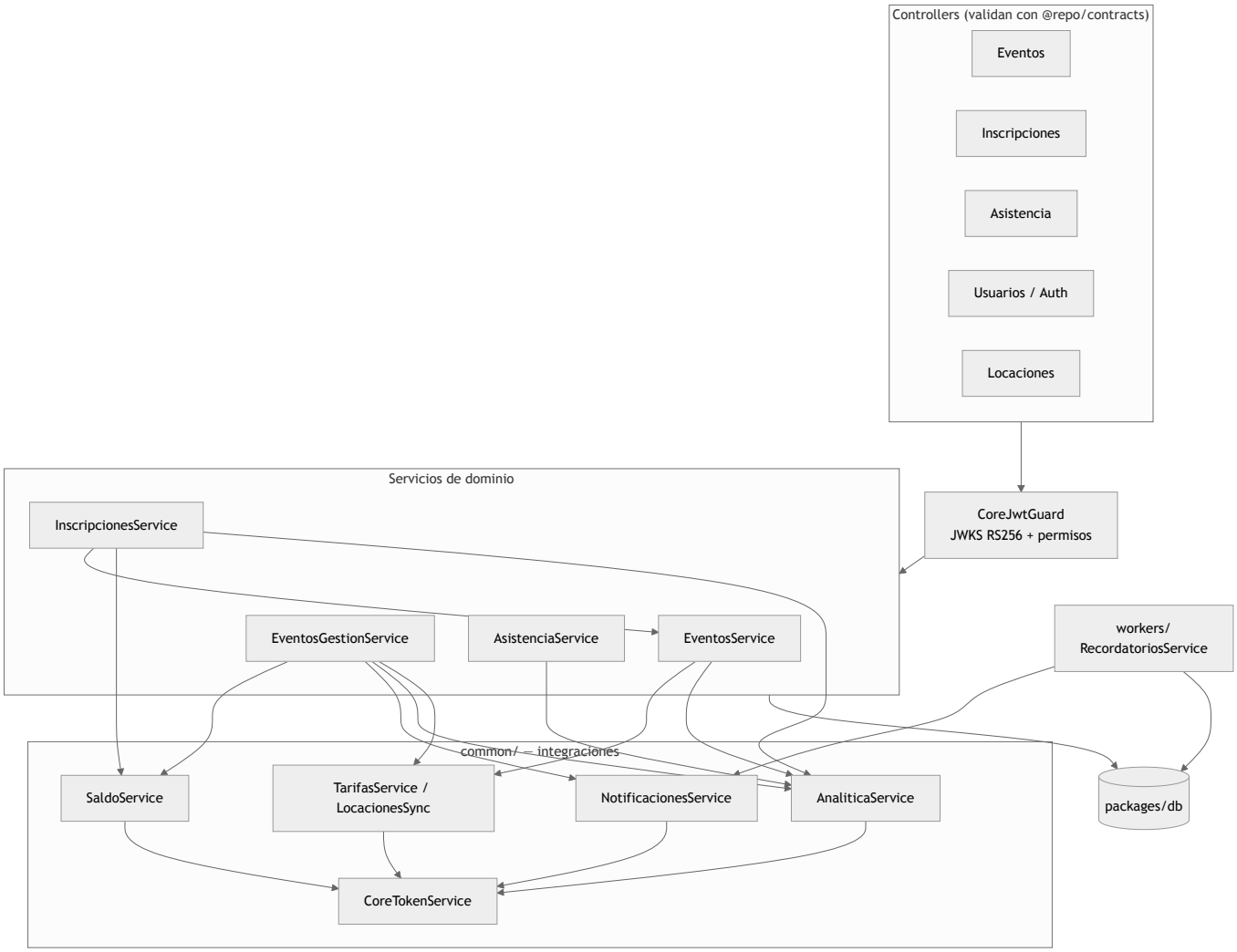


Reglas fijadas por la coordinación de cátedra: identidad, billetera, avisos entre módulos y notificaciones al usuario **pasan siempre por CORE**. Los pedidos sincrónicos de datos a su dueño (sedes, espacios, tarifas, usuarios) van **directo** al módulo dueño, firmados con el token de servicio que emite CORE.

Usuarios y roles

Si el token de CORE trae…	Perfil	Qué ve
rol `OPERATIONS_ADMIN` o `ADMIN`, o el permiso `campus-events:manage`	Administrativo	Gestión, alta de eventos, asistencia y todo lo de los demás perfiles
rol `TEACHER`	Docente	Cartelera, inscripciones, cuenta y avisos
cualquier otro	Estudiante	Cartelera, inscripciones, cuenta y avisos

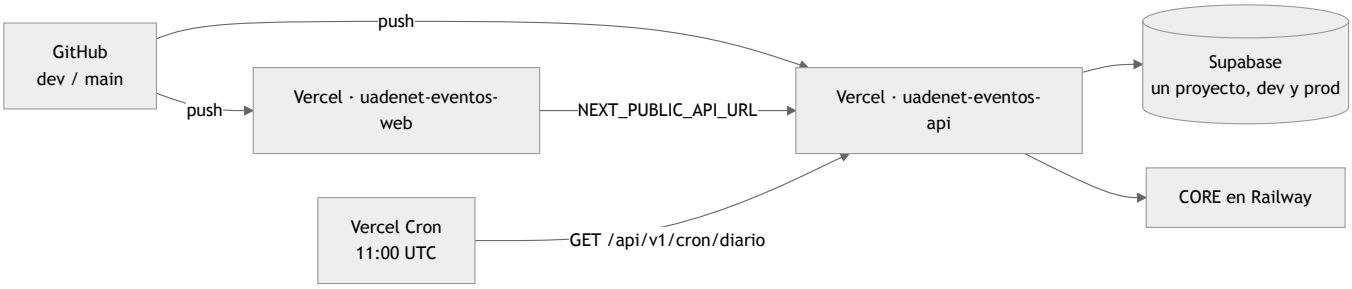
Dentro de `apps/api`



Cuando un módulo tiene una parte que depende de CORE y otra que no, se parte en dos services: `EventosService` (cartelera y alta, sin CORE) y `EventosGestionService` (editar, cancelar, reembolsos y avisos). El worker de recordatorios corre fuera de los requests HTTP.

Toda integración externa vive en `common/` detrás de un service propio, así los módulos de dominio no conocen URLs ni formatos de CORE o Backoffice. Sin `CORE_BASE_URL` / `BACKOFFICE_BASE_URL`, la API monta en el mismo proceso un CORE y un Backoffice simulados con los mismos contratos: el equipo desarrolla y prueba sin depender de que los otros grupos estén arriba.

Despliegue



Rama	Web	API
`dev`	`uadenet-eventos-web-dev.vercel.app`	`uadenet-eventos-api-dev.vercel.app`
`main`	`uadenet-eventos-web.vercel.app`	`uadenet-eventos-api.vercel.app`

Web y API son dos proyectos de Vercel sobre el mismo repositorio. Las ramas de trabajo salen de `dev` y entran por PR con revisión; `dev` se promueve a `main` antes de cada entrega. Dev y prod comparten la base de Supabase, así que una migration afecta a los dos y se avisa al equipo antes de correrla.

Como Vercel es serverless, los procesos periódicos (recordatorios y sincronización de locaciones) no pueden depender de `setInterval`: un **cron diario** de Vercel llama a `/api/v1/cron/diario`, protegido con `CRON_SECRET`. En local, con la API como proceso largo, siguen corriendo los intervalos.

Seguridad

- **Autenticación:** `CoreJwtGuard` valida en cada request la firma RS256 y el vencimiento del token contra el JWKS de CORE (`/.well-known/jwks.json`) y, si está configurado, la firma `X-Core-Signature` del gateway.
- **Autorización en dos capas:** el permiso de CORE `campus-events:manage` para gestionar, y la regla de pertenencia en el service: solo el creador edita, cancela o toma asistencia de su evento.
- **Validación en el borde:** todo request se valida con los schemas Zod de `@repo/contracts` antes de llegar al service.
- **Dinero:** débitos con `Idempotency-Key`; antes de revertir se consulta el estado del cobro en CORE para no devolver dos veces.

Integraciones con terceros

Otros módulos de UADEnet

Con quién	Para qué	Cómo
CORE (grupo 10)	Identidad: login, refresh, logout y perfil	`POST /api/v1/auth/login`, `refresh`, `logout`, `GET /auth/me`. Emite JWT RS256 con roles y permisos (`AuthModule`).
	Validar el token en cada request	JWKS de CORE (`CoreJwksService` + `CoreJwtGuard`); hace upsert del usuario en la tabla local `usuarios`.
	Token de servicio (máquina a máquina)	`POST /internal/v1/service-auth/token` como `events-service`; se cachea ~15 min y firma las llamadas a CORE y Backoffice (`CoreTokenService`).
	Billetera: saldo, cobro, reembolso	`GET /wallets/me`; débito `POST /internal/v1/wallets/{userId}/debits` con `Idempotency-Key`; reversa `POST /internal/v1/wallet-transactions/{id}/reversals`; conciliación por referencia (`SaldoService`).
	Notificaciones al usuario	`POST /internal/v1/notifications`: recordatorio a 7 días, reprogramación y cancelación (`NotificacionesService` + worker de recordatorios).
	Bus de eventos de dominio	`POST /internal/v1/events` con el sobre estándar v1; CORE los reparte por RabbitMQ.
Backoffice (grupo 8)	Sedes, espacios aptos para eventos, tarifas, datos de usuario	Llamada directa con el token de servicio. Sedes y espacios se sincronizan en la tabla `locaciones`; la tarifa vigente se guarda en una caché en memoria refrescada por polling, así una caída de Backoffice no frena cada inscripción (ADR 0013, 0014).
Analítica Institucional	Medir el uso del módulo	Recibe por el bus de CORE: `evento.creado`, `evento.modificado`, `evento.cancelado`, `inscripcion.registrada`, `asistencia.confirmada`, validados con Zod antes de publicarse (`AnaliticaService`).
Portales Estudiante / Docente	Mostrar eventos y avisos	Consumen `GET /api/v1/eventos` y `/eventos/{id}`; reciben los avisos con link directo al evento en la campanita de CORE.

Servicios externos de infraestructura

Servicio	Uso
Supabase	PostgreSQL administrado. Solo se usa la base: la app se conecta por el pooler de transacciones (`prepare: false`) y las migrations, por conexión directa. No se usan Auth, Storage ni Realtime de Supabase: la identidad es de CORE.
Vercel	Hosting de web y API (funciones serverless), deploy por rama y el cron diario.
GitHub	Repositorio, PRs con revisión obligatoria y CI (lint, typecheck, test, build).
Railway	No lo usamos directamente: es donde el grupo de CORE despliega su API.

Almacenamiento de imágenes y QR

El módulo no usa ningún servicio externo de almacenamiento de imágenes (S3, Supabase Storage, Cloudinary ni similares), porque no maneja archivos subidos por usuarios. La única imagen es el QR de la credencial y nunca se guarda:

- **Generación:** el portal dibuja el QR en el navegador con la librería `qrcode`, a partir del id de la inscripción.
- **Lectura:** en la puerta, `jsQR` decodifica los cuadros de la cámara (`getUserMedia` + `<canvas>`) en el propio dispositivo; solo viaja a la API el código leído (ADR 0017).

Son librerías que corren en el cliente, no servicios de terceros: no suman costo, latencia ni datos personales fuera del sistema.

Decisiones de arquitectura

Cada decisión no trivial queda registrada como ADR en `docs/decisions/`, con su contexto, las opciones consideradas y el motivo. Los ADRs son append-only: si una decisión se revierte, se escribe uno nuevo que referencia al anterior. Abajo, agrupadas por tema, con lo que se eligió, lo que se descartó y por qué.

Stack y estructura

ADR 0001 Stack general: Next.js + NestJS + Drizzle + Postgres + TypeScript en Vercel

Contexto: el enunciado deja libre la tecnología; había que elegir algo moderno, no deprecado y defendible en las entregas.

Por qué: Next.js y Vercel son del mismo equipo y se despliegan sin fricción. TypeScript en las dos apps permite compartir tipos entre front y back. Un monorepo porque el frontend y el backend del módulo están acoplados y se versionan juntos.

ADR 0002 Turborepo + pnpm workspaces

Descartado: Nx, y npm o yarn como gestor.

Por qué: Turborepo se integra con Vercel (caché remota sin configuración) y alcanza para dos apps y un equipo de 5-6 personas; Nx se justifica con decenas de paquetes o varios lenguajes. pnpm tiene el mejor soporte de workspaces.

ADR 0003 NestJS como backend separado del frontend

Descartado: Route Handlers de Next.js (un solo deploy, pero mezcla front y back) y Fastify (liviano, sin estructura impuesta).

Por qué: NestJS impone la arquitectura en capas controller → service → repositorio que trabaja la materia y que se integra por servicios REST. Separa de verdad las responsabilidades de front y back, y genera Swagger casi gratis con `@nestjs/swagger`, que es el contrato que leen los otros grupos.

ADR 0004 Drizzle como ORM

Descartado: Prisma, que suma un paso de `generate` y un runtime más pesado sin beneficio a esta escala.

Por qué: bundle chico (~7 KB) y sin generación de cliente, ideal en serverless; API parecida a SQL con tipado completo; y agnóstico del proveedor de Postgres, lo que después permitió cambiar de proveedor dos veces sin tocar las queries (ADR 0008 y 0011).

ADR 0005 Contratos compartidos con Zod en `packages/contracts`

Descartado: generar tipos desde el spec de OpenAPI (`openapi-typescript`).

Por qué: sin paso de generación que pueda romperse. El mismo schema valida el request en la API (pipe de Nest) y el formulario en el portal, así web y API no se desincronizan. El costo, mantener los schemas a mano, es aceptable para el tamaño del proyecto.

Estilos del frontend

ADR 0010 Estilos con CSS Modules y tokens propios

Descartado: Tailwind (su escala no coincide con la del sistema de diseño y habría que reconfigurarla entera), librerías de componentes como shadcn/ui o MUI (traen su propio lenguaje visual) y estilos inline (sin `:hover` ni media queries).

Por qué: el sistema de diseño es chico y cerrado. CSS Modules viene con Next.js, no suma dependencias, y los tokens viven como variables CSS en un solo archivo (`globals.css`).

Base de datos: de Supabase a Neon y de vuelta

ADR 0006 Un proyecto Supabase compartido para desarrollo

Descartado: Supabase local con Docker, una base por persona.

Por qué: arranca en minutos sin depender de que todo el equipo tenga Docker funcionando. Riesgo aceptado: dos migraciones simultáneas se pueden pisar. Se mitiga con la regla de avisar en el grupo antes de migrar y con un seed para volver a un estado conocido.

ADR 0008 Migración a Neon reemplazado por 0011

Contexto: el equipo se quedó sin proyectos gratis en Supabase. Al revisar la dependencia real se vio que solo se usaba el Postgres: la identidad la maneja CORE (no Supabase Auth) y Realtime nunca se adoptó.

Decisión: pasar a Neon, que además ofrece branching de base nativo (una rama de datos por PR). Sin costo de arquitectura gracias a Drizzle.

ADR 0011 Vuelta a Supabase

Contexto: por preferencia del equipo, sin un problema técnico con Neon.

Decisión e impacto: se vuelve al driver `postgres.js` conectado al pooler de transacciones de Supabase con `prepare: false` (recomendado para serverless) y a la estrategia de un proyecto compartido del ADR 0006. Solo se usa `DATABASE_URL` : no se agregó el SDK de Supabase porque no se usan Auth, Storage ni Realtime.

Lección: como el módulo depende de Postgres y no de un proveedor, cambiar de host dos veces costó solo el driver de conexión.

Ramas y flujo de trabajo

ADR 0007 Un único `main` protegido

reemplazado por 0009

Al principio, para un equipo chico, una rama `dev` parecía un paso de sincronización sin beneficio: alcanzaba con `main` protegido, PRs chicos y CI en verde.

ADR 0009 `dev` como rama de integración

Contexto: con varias historias en paralelo, cada merge dejaba `main` inestable entre entregas y no había separación entre "lo que se integra" y "lo que se muestra".

Decisión: las ramas nacen de `dev` y vuelven a `dev` por PR con al menos una aprobación (forzada en GitHub). `main` recibe solo un PR `dev → main` antes de cada entrega, así siempre hay una versión demostrable. Cada rama despliega su propio entorno en Vercel.

Integración con Backoffice

ADR 0012 Locaciones cargadas desde el panel propio

reemplazado por 0013

Se asumió que la locación podía ser texto cargado por el administrativo al crear el evento, para no depender del catálogo de otro grupo. La consulta formal a Backoffice confirmó lo contrario.

ADR 0013 Backoffice define tarifas por categoría y el catálogo de locaciones

Contexto: las respuestas oficiales del grupo 8 cambiaron dos supuestos: el precio no es libre por evento y las locaciones las administra Backoffice.

Decisión: el evento guarda solo una `categoriaPrecio` (`general` o `especial`). El monto sale de la tarifa vigente de Backoffice y se congela en la inscripción al cobrar (`montoCobrado`), para que un cambio de tarifa no reescriba lo ya cobrado. Las locaciones se sincronizan desde su catálogo con el flag `apto_eventos`.

Por qué: no es una preferencia técnica: es la regla de negocio definida por el dueño del dato.

ADR 0014 Tarifas en caché con polling, no consulta en vivo por request

Descartado: llamar a Backoffice de forma sincrónica en cada inscripción paga y en cada listado.

Por qué: eso ataría la disponibilidad del flujo más usado del módulo a la de otro grupo y le sumaría latencia. Se acepta una ventana chica de desactualización del precio. Si no hay tarifa vigente para una categoría, no se crean eventos ni se confirman inscripciones de esa categoría.

Comportamiento del sistema

ADR 0015 Ciclo de vida del evento: edición, reprogramación y cancelación por su creador

Descartado: el borrado físico (rompe las relaciones con inscripciones y asistencias y borra el rastro de lo reembolsado) y reembolsar desde un proceso en segundo plano (en Vercel las funciones se congelan al responder, y el creador necesita saber en el momento cuántos pagos se devolvieron).

Decisión: baja lógica con `estado` (`publicado` o `cancelado`). Solo el creador edita o cancela, y solo antes del inicio. El cupo no puede bajar de los inscriptos, y con inscriptos no se puede cambiar el precio. Si cambian la fecha o la locación, se revalidan las mismas reglas que en el alta, se avisa a cada inscripto y se le permite cancelar sin el límite de 24 horas. Al cancelar se revierte cada cobro en CORE.

Fallas parciales: la cancelación es idempotente. Repetirla solo reintenta los reembolsos pendientes, y antes de revertir se consulta el estado del cobro en CORE para no devolver dos veces.

ADR 0016 Tareas periódicas con un cron diario de Vercel

Contexto: en Vercel la API es serverless y los `setInterval` no corren, así que el recordatorio a 7 días no salía nunca. El plan gratuito (Hobby) admite un cron por día.

Descartado: un proceso largo en otro servicio (otra cosa más que mantener) y un cron externo como GitHub Actions (depende de un tercero y saca el secreto de Vercel).

Decisión: un cron a las 11:00 UTC llama a `/api/v1/cron/diario`, protegido con `CRON_SECRET`. Sincroniza locaciones, envía los recordatorios en lotes de 5 y espera a que terminen antes de responder. Se eligió una hora lejos de la medianoche UTC por la imprecisión de ±59 minutos de Hobby. Las tarifas quedan fuera porque su caché es por instancia.

ADR 0017 Lectura del QR con la cámara, en el navegador

Descartado: la API nativa `BarcodeDetector` (no existe en Safari de iOS, donde se va a usar) y librerías como `html5-qrcode` o `@zxing/browser` (traen interfaz propia y pesan más).

Decisión: `jsQR` sobre la cámara y un `<canvas>`. La cámara se activa con un gesto del operador, se decodifican unos 8 cuadros por segundo y el código pasa por la misma validación que el código pegado a mano.

Por qué: la imagen no sale del dispositivo (ni a la API ni a un tercero), no suma costo de servidor y funciona igual en todos los navegadores.

Pendientes de integración

- Confirmar con CORE los tipos de notificación `EVENT_RESCHEDULED` y `EVENT_CANCELLED` (hoy los acepta el CORE simulado).
- Confirmar con Analítica los eventos nuevos `evento.modificado` y `evento.cancelado`.
- Pasar del Backoffice simulado al real cuando el grupo 8 publique su API.